TUGAS AKHIR

PENGEMBANGAN FITUR UJIAN DAN DOWNLOAD ASSIGNMENT PADA SHARIF-JUDGE

Christofer Setiawan

NPM: 6182301076

PROGRAM STUDI INFORMATIKA
FAKULTAS SAINS
UNIVERSITAS KATOLIK PARAHYANGAN
2026

FINAL PROJECT

DEVELOPMENT OF EXAM AND ASSIGNMENT DOWNLOAD FEATURES FOR SHARIF-JUDGE



Christofer Setiawan

NPM: 6182301076

DEPARTMENT OF INFORMATICS
FACULTY OF SCIENCES
PARAHYANGAN CATHOLIC UNIVERSITY
2026

ABSTRAK

«Tuliskan abstrak anda di sini, dalam bahasa Indonesia»

Kata-kata kunci: «Tuliskan di sini kata-kata kunci yang anda gunakan, dalam bahasa Indonesia»

ABSTRACT

«Tuliskan abstrak anda di sini, dalam bahasa Inggris»

Keywords: «Tuliskan di sini kata-kata kunci yang anda gunakan, dalam bahasa Inggris»

cfttoctitlename

cftloftitlename

BAB 1

PENDAHULUAN

1.1 Latar Belakang

SharIF-Judge merupakan *website* utama bagi mahasiswa informatika Universitas Katolik Parahyangan untuk berlatih dan mengasah kemampuan pemrograman. *Website* ini memuat berbagai *assignment* dan untuk masing-masing *assignment* memiliki *problem* yang harus diselesaikan mahasiswa sebagai bahan latihan. Saat ini SharIF-Judge melakukan penilaian secara otomatis dengan mencocokan keluaran kode dengan jawaban yang sudah disiapkan oleh pembuat soal dan memberikan nilai berdasarkan keakuratan keluaran yang dihasilkan oleh kode mahasiswa. Selain nilai yang diberikan oleh SharIF-Judge, dosen juga harus menilai kode secara manual untuk menilai logika dan efisiensi kode dari setiap mahasiswa. Namun evaluasi secara manual ini memiliki kendala tersendiri, dosen harus mengunduh dan memeriksa berkas jawaban mahasiswa satu per satu secara manual.

Selain kendala penilaian kode mahasiswa secara manual oleh dosen, terdapat pula beberapa kendala pada sisi mahasiswa. Saat ini SharIF-Judge memerlukan dua *instance* berbeda untuk mengerjakan latihan dan *quiz*. Pemisahan ini dilakukan agar mahasiswa tidak dapat melihat catatan dan hasil latihan yang sudah di kerjakan sebelumnya. Namun, pemisahan ini membuat *maintenance* menjadi kurang efisien dan cukup menyulitkan mahasiswa karena mengharuskan mengetik tautan secara manual untuk masuk ke SharIF-Judge khusus untuk *quiz*. Tidak hanya itu, tampilan antar muka SharIF-Judge saat ini masih minim penunjuk visual. Pada daftar *assignment* tidak ada penanda visual untuk membedakan latihan dan *quiz*, serta kurangnya indikator warna untuk durasi pengerjaan *assignment* yang membuat SharIF-Judge menjadi kurang interaktif.

Berdasarkan permasalahan yang dialami oleh dosen dan mahasiswa, pada tugas akhir ini akan mengembangkan dua fitur utama pada SharIF-Judge untuk membantu mengatasi kendala dosen dan mahasiswa. Fitur pertama menambahkan penanda status visual berupa *flag* dan menambahkan pembatasan akses berbasis IPv4, sehingga *quiz* dan latihan bisa tetap berada pada satu *instance* tanpa harus mengurangi keamanan. Akan dikembangkan pula pembaruan tampilan antarmuka dengan menambahkan emoji untuk membedakan latihan dengan *quiz* dan tiga indikator warna berbeda untuk durasi pengerjaan *assignment*. Fitur kedua yang akan dikembangkan adalah pengunduhan seluruh berkas *assignment* dalam format yang sudah terkompresi zip dengan penamaan file *{id-assignment}-{nama-assignment}-{nama-soal}.zip*.

Dengan adanya penggabungan dua *instance* SharIF-Judge menjadi satu *instance* dan tersedianya fitur mengunduh seluruh assignment secara bersamaan, diharapkan akan memudahkan dalam melakukan *maintenance* dan melakukan penilaian kode secara manual. Selain itu perubahan antarmuka SharIF-Judge diharapkan mampu meningkatkan kejelasan informasi bagi mahasiswa

saat mengerjakan latihan maupun *quiz*.

1.2 Rumusan Masalah

1. Bagaimana mengimplementasikan pembatasan akses berdasarkan *IPv4* dan penanda *quiz* dalam satu *instance*?
2. Bagaimana merancang perubahan tampilan antarmuka agar memiliki status visual berupa emoji dan durasi pengerjaan setiap *assignment* diberikan warna sesuai dengan status *assignment*?
3. Bagaimana cara melakukan pengunduhan berkas *assignment* yang sudah dikompresi dalam bentuk zip dan memiliki penamaan *{id-assignment}-{nama-assignment}-{nama-soal}.zip*.

1.3 Tujuan

1. Mengimplementasikan pembatasan akses berdasarkan *IPv4* dan penanda *quiz* dalam satu *instance*.
2. Mengimplementasikan perubahan tampilan antarmuka agar memiliki status visual berupa emoji dan durasi pengerjaan setiap *assignment* diberikan warna sesuai dengan status *assignment*.
3. Mengimplementasikan pengunduhan berkas *assignment* yang sudah dikompresi menjadi bentuk zip dan memiliki penamaan *{id-assignment}-{nama-assignment}-{nama-soal}.zip*.

1.4 Batasan Masalah

Untuk mempermudah pembuatan template ini, tentu ada hal-hal yang harus dibatasi, misalnya saja bahwa template ini bukan berupa style LaTeX pada umumnya (dengan alasannya karena belum mampu jika diminta membuat seperti itu)

1.5 Metodologi

1. Melakukan studi mengenai pembatasan akses *website* berdasarkan IPv4.
2. Melakukan studi mengenai cara mengunduh banyak file secara bersamaan dan dijadikan format zip.
3. Merancang sistem untuk pembatasan akses berdasarkan IPv4 dan mengunduh banyak file lalu mengubahnya dalam bentuk zip.
4. Mengimplementasikan perubahan sistem pada *website* SharIF-Judge.
5. Melakukan Pengujian dan eksperimen.
6. Menganalisis hasil eksperimen.
7. Menulis dokumen tugas akhir.

1.6 Sistematika Pembahasan

- **Bab 1:** Pendahuluan
  Membahas latar belakang, rumusan masalah, tujuan, batasan masalah, metodologi, dan sistematika pembahasan.
- **Bab 2:** Landasan Teori
  Membahas teori-teori yang berhubungan dengan penelitian ini, yaitu SharIF-Judge, CodeIgniter 3, Twig, IDE, subnetting *IPv4*, mengunduh dan melakukan format file.
- **Bab 3:** Analisis
  Membahas analisis terhadap perangkat lunak SharIF-Judge dan IDE pada SharIF-Judge.
- **Bab 4:** Perancangan
  Membahas perancangan fitur yang akan diimplementasikan pada SharIF Judge.
- **Bab 5:** Implementasi dan Pengujian
  Membahas implementasi fitur pada SharIF Judge dan pengujian yang dilakukan beserta dengan analisis hasil pengujian.
- **Bab 6:** Kesimpulan dan Saran
  Membahas kesimpulan dari penelitian ini dan saran untuk penelitian berikutnya.

BAB 2

LANDASAN TEORI

2.1 Skripsi

Rencananya akan diisi dengan penjelasan umum mengenai buku skripsi.

2.2 LaTeX

Mengapa menggunakan LaTeX untuk buku skripsi dan apa keunggulan/kerugiannya bagi mahasiswa dan pembuat template.

2.3 Template Skripsi FTIS UNPAR

Akan dipaparkan bagaimana menggunakan template ini, termasuk petunjuk singkat membuat referensi, gambar dan tabel. Juga hal-hal lain yang belum terpikir sampai saat ini.

2.3.1 Tabel

Berikut adalah contoh pembuatan tabel. Penempatan tabel dan gambar secara umum diatur secara otomatis oleh LaTeX, perhatikan contoh di file bab2.tex untuk melihat bagaimana cara memaksa tabel ditempatkan sesuai keinginan kita.

Perhatikan bawa berbeda dengan penempatan judul gambar gambar, keterangan tabel harus diletakkan di atas tabel!! Lihat Tabel **??** berikut ini:

Tabel 2.1: Tabel contoh

	v_{start}	$\mathcal{S}_1$	v_{end}
τ_1	1	12	20
τ_2	1		20
τ_3	1	9	20
τ_4	1		20

Tabel **??** dan Tabel **??** berikut ini adalah tabel dengan sel yang berwarna dan ada dua tabel yang bersebelahan.

Tabel 2.2: Tabel bewarna(1)

	v_{start}	$\mathcal{S}_2$	$\mathcal{S}_1$	v_{end}
τ_1	1	5	12	20
τ_2	1	8		20
τ_3	1	2/8/17	9	20
τ_4	1			20

Tabel 2.3: Tabel bewarna(2)

	v_{start}	$\mathcal{S}_1$	$\mathcal{S}_2$	v_{end}
τ_1	1	12	5	20
τ_2	1		8	20
τ_3	1	9	2/8/17	20
τ_4	1			20

2.3.2 Kutipan

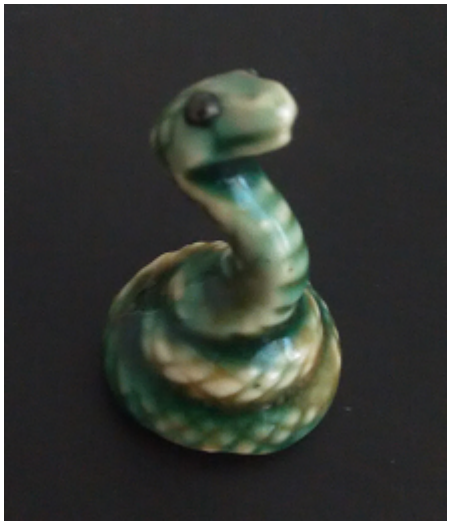
Berikut contoh kutipan dari berbagai sumber, untuk keterangan lebih lengkap, silahkan membaca file referensi.bib yang disediakan juga di template ini. Contoh kutipan:

- Buku: [?]
- Bab dalam buku: [?]
- Artikel dari Jurnal: [?]
- Artikel dari prosiding seminar/konferensi: [?]
- Skripsi/Thesis/Disertasi: [?] [?] [?]
- Technical/Scientific Report: [?]
- RFC (Request For Comments): [?]
- Technical Documentation/Technical Manual: [?] [?] [?]
- Paten: [?]
- Tidak dipublikasikan: [?] [?]
- Laman web: [?]
- Lain-lain: [?]

2.3.3 Gambar

Pada hampir semua editor, penempatan gambar di dalam dokumen LaTeX tidak dapat dilakukan melalui proses *drag and drop*. Perhatikan contoh pada file bab2.tex untuk melihat bagaimana cara menempatkan gambar. Beberapa hal yang harus diperhatikan pada saat menempatkan gambar:

- Setiap gambar **harus** diacu di dalam teks (gunakan *field* LABEL)
- *Field* CAPTION digunakan untuk teks pengantar pada gambar. Terdapat dua bagian yaitu yang ada di antara tanda [dan] dan yang ada di antara tanda { dan }. Yang pertama akan muncul di Daftar Gambar, sedangkan yang kedua akan muncul di teks pengantar gambar. Untuk skripsi ini, samakan isi keduanya.
- Jenis file yang dapat digunakan sebagai gambar cukup banyak, tetapi yang paling populer adalah tipe PNG (lihat Gambar **??**), tipe JPG (Gambar **??**) dan tipe PDF (Gambar **??**)
- Besarnya gambar dapat diatur dengan *field* SCALE.
- Penempatan gambar diatur menggunakan *placement specifier* (di antara tanda [dan] setelah deklarasi gambar. Yang umum digunakan adalah **H** untuk menempatkan gambar **sesuai** penempatannya di file .tex atau **h** yang berarti "kira-kira" di sini.
  Jika tidak menggunakan *placement specifier*, LaTeX akan menempatkan gambar secara otomatis untuk menghindari bagian kosong pada dokumen anda. Walaupun cara ini sangat mudah, hindarkan terjadinya penempatan dua gambar secara berurutan.

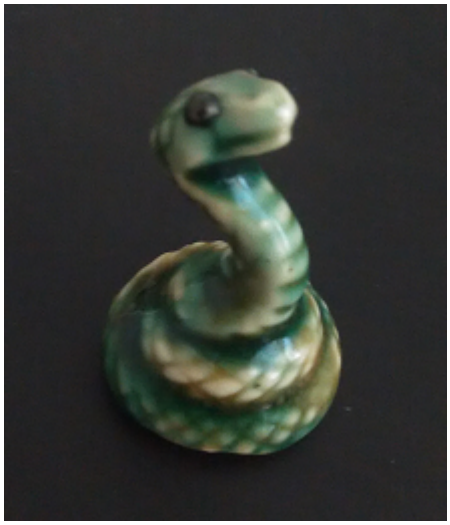


Gambar 2.1: Gambar *Serpentes* dalam format png

- Gambar **??** ditempatkan di bagian atas halaman, walaupun penempatannya dilakukan setelah penulisan 3 paragraf setelah penjelasan ini.
- Gambar **??** dengan skala 0.5 ditempatkan di antara dua buah paragraf. Perhatikan penulisannya di dalam file bab2.tex!
- Gambar **??** ditempatkan menggunakan *specifier* **h**.



Gambar 2.2: Ular kecil



Gambar 2.3: *Serpentes* jantan

2.3.4 Kode Program

Kode program dalam bahasa tertentu seringkali harus ditulis di dalam bab, bukan hanya dilampirkan di bagian Lampiran. Kode **??** menampilkan penggunaan karakter-karakter yang umum digunakan dalam sebuah program yang ditulis dengan bahasa C.

Kode 2.1: Kode untuk menampilkan karakter-karakter aneh

```
// This does not make algorithmic sense,
// but it shows off significant programming characters.

#include<stdio.h>

void myFunction( int input, float* output ) {
    switch ( array[i] ) {
        case 1: // This is silly code
            if ( a >= 0 || b <= 3 && c != x )
                *output += 0.005 + 20050;
            char = 'g';
            b = 2^n + ~right_size - leftSize * MAX_SIZE;
            c = (--aaa + &daa) / (bbb++ - ccc % 2 );
            strcpy(a,"hello $@?");
    }
    count = ~mask | 0x00FF00AA;
}

// Fonts for Displaying Program Code in LaTeX
// Adrian P. Robson, nepsweb.co.uk
// 8 October 2012
// http://nepsweb.co.uk/docs/progfonts.pdf
```

2.3.5 Notasi

Simbol-simbol (matematika) yang sering digunakan sepanjang penulisan skripsi, dapat dimasukkan ke dalam "Daftar Notasi". Daftar ini ada di halaman depan sebelum Bab **??**. Cara memasukkan sebuah simbol ke dalam Daftar Notasi adalah menggunakan perintah `\nomenclature`. Contoh:

```
\nomenclature[]{$A$}{luas kandang ular}
```

Argumen opsional digunakan untuk mengurutkan notasi. Silahkan lihat sendiri dokumentasi package `nomencl`

BAB 3

ANALISIS

3.1 Analisis Sistem Kini

SharIF-Judge adalah tempat penilaian kode mahasiswa yang telah dimodifikasi untuk menyesuaikan dengan kebutuhan yang diperlukan oleh jurusan Informatika Universitas Katolik Parahyangan. SharIF-Judge menerapkan arsitektur *Model-View-Controller* (MVC). Berikut merupakan hasil eksplorasi dari situs web SharIF-Judge:

3.1.1 Model

Model merupakan kelas-kelas yang digunakan untuk mengelola data dan mengembalikan data dari database. Kumpulan kelas model tersimpan pada direktori `application/models`. Dibawah ini merupakan penjelasan dari setiap kelas model dan fungsinya pada SharIF-Judge.

- `Assignment_model.php`
  `Assignment_model.php` berfungsi untuk mengelola tabel `assignments` dan mengembalikan data dari database untuk digunakan pada halaman *assignment*. Fungsi `Assignment_model.php` yaitu:
  - `add_assignment($id, $edit)`
    Membuat atau memperbaharui pada sebuah *assignment*.
  - `delete_assignment($assignment_id)`
    Menghapus *assignment*.
  - `all_assignments()`
    Menampilkan daftar seluruh *assignment* berserta dengan informasi yang dimiliki setiap *assignment*.
  - `new_assignment_id()`
    Mengembalikan nomor paling kecil dan digunakan sebagai *id assignment* baru.
  - `all_problems($assignment_id)`
    Mengembalikan *problems* dari *assignment*.
  - `problem_info($assignment_id, $problem_id)`
    Mengembalikan seluruh informasi mengenai *problem*
  - `assignment_info($assignment_id)`
    Mengembalikan seluruh informasi sebuah *assignment*
  - `is_participant($participants, $username)`
    Mengecek apakah suatu `$username` terdapat pada kumpulan `$participants` dan meng-

205 embalikan nilai *boolean*.
206 – `increase_total_submits($assignment_id)`
207 Meningkatkan jumlah *submits* yang dapat dilakukan pada sebuah *assignment*.
208 – `set_moss_time($assignment_id)`
209 Memperbaharui "*Moss Update Time*" pada *assignment*.
210 – `get_moss_time($assignment_id)`
211 Mendapatkan dan mengembalikan nilai "*Moss Update Time*" pada sebuah assignment.
212 – `save_problem_description($assignment_id, $problem_id, $text, $type)`
213 Menambahkan atau memperbaharui deskripsi pada sebuah *problem* pada sebuah *assignment*.
215 – `_update_coefficients($a_id, $extra_time, $finish_time, $new_late_rule)`
216 Memperbaharui koefisien dari sebuah *assignment*.
217 • `Hof_model.php`
218 `Hof_model.php` berfungsi untuk mengembalikan informasi yang akan di tampilkan pada *hall of fame* yang didapatkan dari tabel `submissions`. Fungsinya adalah sebagai berikut:
220 – `get_all_final_submission()`
221 Mendapatkan dan mengembalikan total nilai dari seluruh *final submission* untuk seluruh pengguna.
223 – `get_all_user_assignments($username)`
224 Mendapatkan dan mengembalikan nilai untuk pengguna tertentu.
225 • `Logs_model.php`
226 `Logs_model.php` berfungsi untuk mengelola pencatatan `logins`. Fungsi yang dimiliki model ini adalah sebagai berikut:
228 – `insert_to_logs($username, $ip_address)`
229 Mencatat waktu *login* pengguna dan menghapus catatan *login* jika melebihi 24 jam.
230 – `get_all_logs()`
231 Mendapatkan dan mengembalikan semua catatan *login*.
232 • `Notifications_model.php`
233 `Notifications_model.php` berfungsi untuk mengelola `notification`. Fungsi yang di miliki adalah sebagai berikut:
235 – `get_all_notifications()`
236 Mendapatkan dan mengembalikan semua *notifications*.
237 – `get_latest_notifications()`
238 Mendapatkan dan mengembalikan 10 *notifications* terbaru.
239 – `add_notification($title, $text)`
240 Menambahkan *notification* baru.
241 – `update_notification($id, $title, $text)`
242 Melakukan perubahan dan Memperbaharui *notification*.
243 – `delete_notification($id)`
244 Menghapus sebuah *notification*.
245 – `get_notification($notif_id)`
246 Mendapatkan dan mengembalikan *notification*.

- `have_new_notification($time)`
  Mengecek apakah terdapat *notification* baru dan mengembalikan nilai bertipe *boolean*.

- `Queue_model.php`
  `Queue_model.php` berfungsi untuk mengelola tabel `queue` dan menampilkan datanya. Fungsi yang dimiliki adalah sebagai berikut:
  - `in_queue($username, $assignment, $problem)`
    Mengecek apakah ada pengguna yang masih memiliki *queue* dalam sebuah *problem* dan mengembalikan nilai *boolean*.
  - `get_queue()`
    Mendapatkan dan mengambil seluruh *submission queue*.
  - `empty_queue()`
    Menghapus seluruh *queue*.
  - `add_to_queue($submit_info)`
    Menambahkan *submission* baru kedalam *queue*.
  - `rejudge($assignment_id, $problem_id)`
    Berfungsi untuk melakukan proses penilaian ulang dengan menambahkan seluruh *submission* kedalan *queue*.
  - `rejudge_single($submission)`
    Melakukan proses penilaian ulang hanya untuk beberapa *submission* tertentu.
  - `get_first_item()`
    Mengambalikan yang berada pada urutan pertama pada *queue*.
  - `remove_item($username, $assignment, $problem, $submit_id)`
    Menghapus sebuah *item* dalam `queue`.
  - `save_judge_result_in_db ($submission, $type)`
    Menyimpan hasil penilaian *judge* ke dalam *database*.
  - `add_to_queue_exec($submit_info)`
    Menambahkan sebuah *dummy submission* dari IDE yang digunakan hanya untuk dijalankan ke dalam *queue*.

- `Scoreboard_model.php`
  `Scoreboard_model.php` berfungsi untuk mengelola tabel *scoreboard*. Fungsi yang dimiliki adalah sebagai berikut:
  - `_generate_scoreboard($assignment_id)`
    Menghasilkan *scoreboard* untuk sebuah *assignment* dari nilai akhir seluruh *submission*.
  - `update_scoreboards()`
    memperbaharui *scoreboard* untuk seluruh *assignment*.
  - `update_scoreboard($assignment_id)`
    Memperbaharui *scoreboard* untuk *assignment*.
  - `get_scoreboard($assignment_id)`
    Mendapatkan dan mengembalikan *scoreboard* pada *assignment*.

- `Settings_model.php`
  `Settings_model.php` berfungsi untuk mengelola tabel *settings*. Fungsi yang dimiliki adalah sebagai berikut:

  - `get_setting($key)`
    Mendapatkan dan mengembalikan nilai dari sebuah `$key` pada tabel `settings`.
  - `set_setting($key, $value)`
    Melakukan perubahan dan memperbaharui nilai dari pada *setting* `$key`.
  - `get_all_settings()`
    Mendapatkan dan Mengembalikan seluruh *settings*.
  - `set_settings($settings)`
    Memperbaharui seluruh nilai perubahan *settings*.
- `Submit_model.php`
  `Submit_model.php` berfungsi untuk mengelola tabel *submission*, Fungsi yang dimiliki adalah sebagai berikut:
  - `get_submission($username, $assignment, $problem, $submit_id)`
    Mendapatkan dan meengembalikan baris data *submission* tertentu.
  - `get_final_submissions($a_id, $u_vl, $uname, $p_num, $fil_u, $fil_prob)`
    Mendapatkan dan mengembalikan seluruh *final submission* pada *assignment*. Pengguna yang memiliki role *student* hanya dapat melihat *final submission* dirinya sendiri.
  - `get_all_submissions($a_id, $u_vl, $uname, $p_num, $fil_u, $fil_prob)`
    Mendapatakan dan mengembalikan seluruh *submission* pada *assignment*. Pengguna yang memiliki role *student* hanya dapat melihat *submission* dirinya sendiri.
  - `count_final_submissions($a_id, $u_vl, $uname, $fil_u, $fil_prob)`
    Menghitung jumlah *final submission* pada *assignment*.
  - `count_all_submissions($a_id, $u_vl, $uname, $fil_u, $fil_prob)`
    Menghitung total seluruh jumlah *submission* pada *assignment*.
  - `set_final_submission($username, $assignment, $problem, $submit_id)`
    Memperbaharui sebuah *submission* menjadi *final submission*.
  - `add_upload_only($submit_info)`
    Menyimpan hasil *upload only problem* ke dalam tabel *database*.
- `User.php`
  `User.php` berfungsi untuk menyimpan *settings* pengguna. Fungsi yang dimiliki adalah sebagai berikut:
  - `select_assignment($assignment_id)`
    Mengambil *assignment* yang dipilih oleh pengguna dan menyimpannya.
  - `save_widget_positions($positions)`
    Menyimpan posisi *widget* sebuah pengguna.
  - `get_widget_positions()`
    Mendapatkan dan mengembalikan posisi *widget* sebuah pengguna.
- `User_model.php`
  `User_model.php` berfungsi untuk mengelola tabel *users*. Fungsi yang dimiliki adalah sebagai berikut:
  - `have_user($username)`
    Memeriksa apakah *username* sudah berada pada database, dan mengembalikan nilai boolean.

- `user_id_to_username($user_id)`
  Mengembalikan *username* dari `$user_id` yang dipilih.
- `username_to_user_id($username)`
  Mengembalikan *user id* dari `$username` yang dipilih.
- `have_email($email, $username)`
  Memeriksa jika pengguna yang dipilih memiliki email pada database dan mengembalikan nilai *boolean*
- `add_user($username, $email, $display_name, $password, $role)`
  Menambahkan satu buah pengguna baru ke dalam *database*.
- `add_users($text, $send_mail, $delay)`
  Menambahkan banyak pengguna baru ke dalam *database*.
- `delete_user($user_id)`
  Menghapus satu buah pengguna dalam *database*.
- `delete_submissions($user_id)`
  Menghapus seluruh *submissions* yang sudah di *submit* oleh sebuah pengguna.
- `validate_user($username, $password)`
  Memeriksa *password* dan *username* pengguna dan mengembalikan nilai *boolean*
- `selected_assignment($username)`
  Mendapatkan dan mengembalikan *assignment* yang dipilih oleh `$username`.
- `get_names()`
  Mendapatkan dan mengembalikan seluruh *display name* pada tabel *users*.
- `update_profile($user_id)`
  Memperbaharui nama, email, password, atau role pengguna.
- `send_password_reset_mail($email)`
  Mengirimkan *link reset password* ke email pengguna yang dapat dipakai selama 1 jam.
- `passchange_is_valid($passchange_key)`
  Memeriksa jika *link reset password* masih dapat digunakan oleh pengguna atau tidak, dan akan mengembalikan nilai bertipe *boolean*
- `reset_password($passchange_key, $newpassword)`
  Memperbaharui *password* dan melakukan validasi dengan*password change key*.
- `get_all_users()`
  Mendapatkan dan mengembalikan seluruh pengguna pada tabel `users`.
- `get_user($user_id)`
  Mendapatkan dan mengembalikan sebuah pengguna yang memiliki id `$user_id`.
- `update_login_time($username)`
  Memperbaharui catatan *login* untuk sebuah pengguna.

3.1.2 View

View adalah yang menjadi tampilan dan yang berinteraksi langsung dengan pengguna dan sisten. Pada SharIF-Judge seluruh file *view* disimpan pada direktori `application/views`, didalam direktori tersebut memiliki 3 sub direktori terpisah yaitu *pages, errors, template*.

- `pages`

Pada direktori *pages*, terdapat tampilan halaman utama. Pada direktori *pages* terdapat 2 sub direktori lainnya.

- `pages/admin`
  Direktori *admin* yang berisikan khusus untuk *role admin*. Berikut merupakan *views* pada direktori *admin*:
  - `add_assignment.twig`
  - `add_notification.twig`
  - `add_user.twig`
  - `add_user_result.twig`
  - `delete_assignment.twig`
  - `edit_problem_html.twig`
  - `edit_problem_md.twig`
  - `edit_problem_plain.twig`
  - `install.twig`
  - `logs.twig`
  - `moss.twig`
  - `queue.twig`
  - `rejudge.twig`
  - `settings.twig`
  - `users.twig`
- `pages/authentication`
  Direktori *authentication* berisi halaman khusus untuk bagian *authentication*. Berikut merupakan *views* pada direktori `admin`:
  - `login.twig`
  - `lost.twig`
  - `register.twig`
  - `register_success.twig`
  - `reset_password.twig`
- `assignments.twig`
- `dashboard.twig`
- `halloffame.twig`
- `notification.twig`
- `problems.twig`
- `profile.twig`
- `scoreboard.twig`
- `scoreboard_tabel.twig`
- `submissions.twig`
- `submit.twig`

- `errors`
  Direktori *errors*, berisi tampilan halaman *error* jika terjadi error. Berikut merupakan *views* pada direktori `errors`:
  - `error_404`

- `error_db`
- `error_exception`
- `error_general`
- `error_php`

- `templates`
  Direktori *templates*, berisi tampilan yang akan digunakan secara berulang seperti *header*, *sidebar*, dan *base*. Berikut merupakan *views* pada `templates`:
  - `base.twig`
  - `side_bar.twig`
  - `simple_header.twig`
  - `top_bar.twig`

3.1.3 Controller

Berikut merupakan file *controller* dan penjelasan fungsinya pada SharIF Judge:

- `Assignments.php`
  Berikut fungsi dengan penjelasannya pada *controller* `Assignments.php`:
  - `select()`
    Memilih *assignment* yang ditampilkan pada *top bar* menggunakan *ajax request*.
  - `pdf($assignment_id, $problem_id, $no_download)`
    Mengunduh *assignment* atau *problem* dalam bentuk PDF ke browser.
  - `downloadtestsdesc($assignment_id)`
    Mengunduh dan mencompress data uji dan deskripsi sebuah *assignment*.
  - `download_submissions($type, $assignment_id)`
    Mengunduh semua *final submission* pada semua *assignment*.
  - `delete($assignment_id)`
    Menghapus sebuah *assignment*.
  - `add()`
    Mendapatkan *input* dari pengguna untuk menambah atau memperbaharui *assignment*.
  - `_add()`
    Menambahkan atau memperbaharui sebuah *assignment*.
  - `edit($assignment_id)`
    Menandai *assignment* yang akan di *edit* dan memanggil fungsi `add`.
  - `pdfCheck($assignment_id, $problem_id)`
    Melakukan validasi ketersediaan pdf pada sebuah *assignment* atau pada sebuah *problem*.
  - `index()`
    Mengambil data dari `Assignment_model` dan menyimpan data dan mengembalikan *views* `assignments.twig`.
- `Dashboard.php`
  Berikut fungsi dengan penjelasannya pada *controller* `Dashboard.php`:
  - `widget_positions()`

menggunakan *ajax request* untuk menyimpan posisi *widget*.

* `index()`
  Mendapatkan data dari beberapa model yaitu `Assignment_model`, `Settings_model`, `User`, dan `Notifications_model`. Data akan dimasukkan ke dalam `dashboard.twig` yang akan dikembalikan ke pengguna.

- `Halloffame.php`
  Berikut fungsi dengan penjelasannya pada *controller* `Halloffame.php`:
  * `hof_details()`
    Menampilkan nilai akhir semua *problem* dan *assignments* pada sebuah pengguna.
  * `index()`
    Mendapatkan data dari `Hof_model` dan mengembalikan *view* `halloffame.twig`. Gambar **??** menunjukkan hasil halaman Hall of Fame yang dapat diakses oleh semua *role*.
- `Install.php`
  Pada *controller* `Install.php` hanya ada satu fungsi yang menangani pembuatan seluruh tabel pada *database* yang dibutuhkan oleh SharIF Judge. Setelah membuat *database* akan mengembalikan *view* `install.twig` yang dapat diisi oleh pengguna tentang data pengguna dengan role *admin* saat *form* di kirim.
- `Login.php`
  Berikut fungsi dengan penjelasannya pada *controller* `Login.php`:
  * `index()`
    Mengembalikan *view* `login.twig` dan memeriksa username dan password pada *form* saat di kirim.
  * `_registration_code($code)`
    Melakukan validasi kode registrasi.
  * `register()`
    Menunjukkan halaman `register.twig` dan membuat pengguna baru.
  * `logout()`
    Melakukan *Log out* dan mengalihkan ke halaman *login*.
  * `lost()`
    Mengirimkan email *reset password*.
  * `reset($passchange_key)`
    Melakukan *reset password* dengan halaman `reset_password.twig`.
- `Logs.php`
  Pada *controller* `Logs.php` hanya memiliki satu fungsi yaitu `index()`, dimana fungsi tersebut akan mendapatkan data dari `Logs_model` dan menampilkan halaman `logs.twig`.
- `Moss.php`
  Berikut fungsi dengan penjelasannya pada *controller* `Moss.php`:
  * `index()`
    Mengambil data dan memasukkannya ke dalam *view* `moss.twig`.
  * `update($assignment_id)`
    Memperbaharui *settings* dari masukkan `moss_userid` pengguna.

497 * `_detect($assignment_id)`
498 Melakukan pemeriksaan kesamaan kode dengan Moss.
499 – `Notifications.php`
500 Berikut fungsi dengan penjelasannya pada *controller* `Notifications.php`:
501 * `add()`
502 Menambahkan atau memperbaharui sebuah *notification*.
503 * `edit($notif_id)`
504 Menandai *notification* yang akan di *edit* dan memanggil fungsi `add`.
505 * `delete()`
506 Menghapus sebuah *notification*.
507 * `check()`
508 menggunakan *ajax request* untuk mengetahui ketersediaan *notification* baru.
509 * `index()`
510 Mendapatkan data dari dua model yaitu `Assignment_model` dan `Notifications_model`.
511 Data akan dimasukkan ke dalam *view* `notifications.twig` yang akan dikembalikan
512 ke pengguna.
513 – `Problems.php`
514 Berikut fungsi dengan penjelasannya pada *controller* `Notifications.php`:
515 * `edit()`
516 Memperbaharui deskripsi sebuah *problem* dalam bentuk `html` atau `markdown`.
517 * `index()`
518 Mendapatkan data *problem* dari berbagai *model* sesuai dengan *assignment* yang
519 dipilih dan menyimpan data tersebut pada halaman `problems.twig` yang akan
520 ditampilkan ke pengguna.
521 – `Profile.php`
522 Berikut fungsi dengan penjelasannya pada *controller* `Profile.php`:
523 * `index()`
524 Mendapatkan data dari berbagai *model* terutama dari pengguna yang akan dima-
525 sukkan ke dalam *view* `profile.twig`. Fungsi ini juga menangani pengiriman *form*
526 pembaharuan data pengguna pengguna.
527 * `_password_check($str)`
528 Melakukan validasi *input password*.
529 * `_password_again_check($str)`
530 Melakukan validasi *input* tulisan pengulangan *password*.
531 * `_email_check($str)`
532 Melakukan validasi ketersediaan email pada *database*.
533 * `_role_check($str)`
534 Melakukan validasi *role* pengguna saat ingin mengubah *role* pengguna.
535 – `Queue.php`
536 Berikut fungsi dengan penjelasannya pada *controller* `Profile.php`:
537 * `pause()`
538 Memberhentikan proses *queue*.

    * `resume()`
      Melanjutkan proses *queue*.
    * `empty_queue()`
      Menghapus semua *queue* yang ada.
    * `index()`
      Mendapatkan data dari *model* `Queue`, `Assignments_model`, dan `Settings_model` yang dipakai dalam *view* `queue.twig` dan ditampilkan kepada pengguna.
- `Queueprocess.php`
  Controller `Queueprocess.php` hanya memiliki satu fungsi yaitu `run()` yang akan menjalankan *queue* satu per satu menggunakan `bash`.
- `Rejudge.php`
  Berikut fungsi dengan penjelasannya pada *controller* `Profile.php`:
    * `rejudge_single()`
      Melakukan *rejudge* untuk satu buah *submission*.
    * `index()`
      Mendapatkan data dan menampilkan `rejudge.twig`. Fungsi ini juga dapat melakukan *rejudge* pada sebuah *problem* tertentu.
- `Scoreboard.php`
  Controller `Queueprocess.php` hanya memiliki satu fungsi yaitu `index()` yang akan menampilkan *view* `scoreboard.twig` dengan data yang didapat dari model `Scoreboard_model`.
- `Server_time.php`
  Controller `Queueprocess.php` hanya memiliki satu fungsi yaitu `index()` yang akan mencetak waktu pada *server*, waktu akan digunakan untuk sinkronisasi waktu.
- `Settings.php`
  Berikut fungsi dengan penjelasannya pada *controller* `Settings.php`:
    * `update()`
      Memperbaharui *settings* dari masukkan pengguna.
    * `index()`
      Mendapatkan data dari `Settings_model` dan menampilkan *view* `settings.twig`. Jika terdapat *error setting* pada sistem, akan ditampilkan juga pada *view* tersebut.
- `Submissions.php`
  Berikut fungsi dengan penjelasannya pada *controller* `Submissions.php`:
    * `all()`
      Fungsi ini akan mendapatkan data dari `Submit_model` untuk mendapatkan seluruh *submission* dan menampilkan halaman `submission.twig` berisi semua *submission*.
    * `the_final()`
      Fungsi ini akan mendapatkan data dari `Submit_model` untuk mendapatkan *final submission* dan menampilkan halaman `submission.twig` berisi *final submission*.
    * `_download_excel($view)`
      Fungsi ini menggunakan *library* PHPExcel untuk membuat sebuah *file excel* dari *submissions* yang akan diunduh pengguna.
    * `final_excel()`

Menggunakan fungsi `_download_excel` untuk mendownload *final submission*.

* `all_excel()`
  Menggunakan fungsi `_download_excel` untuk mendownload seluruh *submission*.
* `select()`
  Fungsi ini digunakan sebagai fungsi yang dipanggil oleh *ajax request* untuk memilih *submission* yang akan dikumpulkan atau menjadi *final*.
* `_check_type($type)`
  Melakukan validasi tipe *submission* yang dikumpulkan.
* `view_code()`
  Digunakan untuk melihat kode, melihat hasil kode, atau melihat *log* sebuah *submission*.
* `download_file()`
  Mengunduh *file* kode sebuah *submission*.

– `Submit.php`

Berikut fungsi dengan penjelasannya pada *controller* `Submit.php`:

* `_language_to_type($language)`
  Mengembalikan kode singkat dari `$language` dipilih.
* `_language_to_ext($language)`
  Mengembalikan extensi file dari `$language` yang dipilih.
* `_match($type, $extension)`
  Melakukan validasi untuk `$type` dan `$extension` agar sesuai.
* `_check_language($str)`
  Melakukan validasi sudah dipilihannya bahasa.
* `_upload()`
  Menyimpan jawaban pengguna yang dikirim dan menambahkannya ke dalam *queue* untuk dinilai jika bukan *upload only problem*.
* `load($problem_id)`
  Mendapatkan isi file dan menyimpan isi file ke editor kode.
* `save($type)`
  Fungsi ini digunakan untuk menyimpan isi editor kode ke dalam *server* dan menjalankan atau mengumpulkan jika diinginkan.
* `_submit($data, $problem_id, $language, $user_dir)`
  Menambahkan kode ke dalam *submission* untuk dinilai.
* `_execute($data, $problem_id, $language, $user_dir)`
  Menambahkan kode ke dalam *queue* untuk dijalankan saja.
* `get_output($problem_id)`
  Mendapatkan keluaran dari kode yang telah dijalankan sebagai hasil eksekusi.
* `index()`
  Fungsi ini akan mendapatkan data dari *model* `Assignments_model` untuk mendapatkan *problem* dan data lainnya. Semua data tersebut akan dimasukkan dalam *view* `submit.twig`.

– `User.php`

Berikut fungsi dengan penjelasannya pada *controller* `User.php`:

* `add()`
  Menambahkan pengguna baru ke dalam *sistem*.
* `delete()`
  Menghapus sebuah pengguna.
* `delete_submissions()`
  Menghapus seluruh *submissions* dari sebuah pengguna.
* `list_excel()`
  Menggunakan *library* PHPExcel untuk membuat sebuah file excel dari seluruh daftar pengguna yang akan diunduh pengguna.
* `index()`
  Fungsi ini akan mendapatkan data dari `User_model` dan menunjukkan *view* `users.twig` menggunakan data tersebut.

3.2 Analisis Kebutuhan Perangkat Lunak

Berdasarkan hasil diskusi dan analisis dengan calon pengguna, terdapat beberapa fitur yang ingin ditambahkan ke dalam SharIF-Judge. Fitur-fitur tersebut dirancang dalam bentuk *Functional Requirements Specification* (FRS) dan untuk masing-masing FRS diturunkan menjadi beberapa *Software Requirements Specification* (SRS).

Functional Requirements Specification (FRS)

Functional Requirements Specification (FRS) merupakan fungsi-fungsi utama dari sudut pandang pengguna yang akan ditambahkan ke dalam aplikasi.

- **FRS001** SharIF-Judge memiliki fitur kuis (*quiz*), untuk memungkinkan satu *instance judge* dapat digunakan untuk latihan dan kuis sekaligus. *Keterangan:* Sebelumnya diperlukan dua *instance* terpisah (contoh: *alpro-quiz.ftis.unpar.ac.id* dan *alpro-lat.ftis.unpar.ac.id*). Jika tidak dipisahkan, saat kuis berlangsung mahasiswa dapat melihat soal-soal latihan.
- **FRS002** SharIF-Judge dapat memberikan akses kepada dosen untuk mengunduh seluruh soal dan jawaban mahasiswa dengan nama file yang sesuai dengan nomor urut *assignment*, nama *assignment*, dan nama soal.

Software Requirements Specification (SRS)

Software Requirements Specification (SRS) merupakan kumpulan kebutuhan spesifik, baik dalam bentuk fungsional maupun non-fungsional, yang akan diimplementasikan ke dalam sistem.

- **SRS001:** Setiap *assignment* memiliki *flag* tambahan "quiz", yang menandakan bahwa *assignment* ini dipersiapkan untuk kuis. Nilai bawaannya (*default*) adalah "latihan" (bukan kuis).
- **SRS002:** Setiap *assignment* (baik kuis maupun latihan) memiliki kolom (*field*) tambahan "IPv4 subnets" yang dapat diisi nilai-nilai terpisah koma dalam format subnet IPv4, seperti *192.0.2.0/24*. Nilai bawaannya adalah kosong.

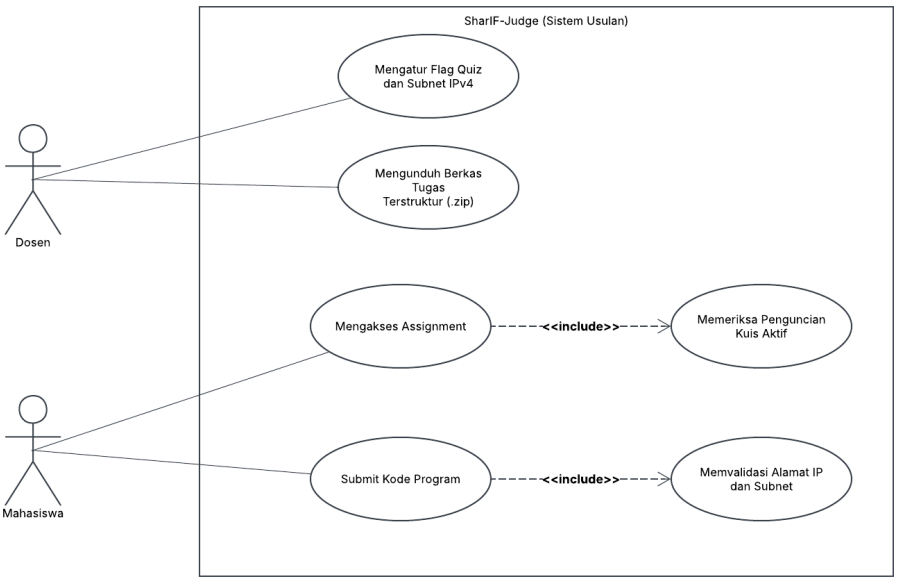
- **SRS003:** Setiap *assignment*, baik kuis maupun latihan, hanya dapat diakses (membaca soal dan mengumpulkan jawaban) dari alamat IP yang terdaftar pada subnet yang sesuai, kecuali jika nilai subnetnya kosong.
- **SRS004:** Pada halaman daftar *assignment*, ditampilkan informasi berupa ikon yang menunjukkan *flag* kuis/latihan. Ikon menggunakan emoji (*Latihan*) dan (*Kuis*).
- **SRS005:** Pada halaman daftar *assignment*, ditampilkan informasi berupa waktu mulai dan waktu selesai *assignment* tersebut. Jika waktu saat ini berada dalam rentang mulai dan selesai sebuah *assignment*, maka informasi ini diwarnai hijau.
- **SRS006:** Jika waktu saat ini berada dalam rentang mulai dan selesai sebuah *assignment* bertipe kuis, maka *assignment-assignment* lain tidak dapat diakses dan informasi sebagaimana dijelaskan pada SRS005 diwarnai merah.
- **SRS007:** Jika waktu saat ini berada dalam rentang mulai dan selesai sebuah *assignment* bertipe latihan, maka *assignment-assignment* lain tetap dapat diakses dan diwarnai hitam.
- **SRS008:** Setiap penolakan akibat aturan-aturan di atas harus disertai pesan penolakan yang jelas, seperti: *"Anda tidak dapat mengakses assignment ini karena mengakses dari IP 192.168.100.176, di luar dari subnet yang diizinkan 10.101.70.118"*.
- **SRS009:** Fitur "*download all assignments*" harus dapat diakses oleh pengguna dengan peran (*role*) *Teacher*.
- **SRS010:** Fitur "*download all assignments by problem*" mengumpulkan seluruh berkas *.zip* hasil unduhan dari fungsi "*Download Final Submissions (by problem)*".
- **SRS011:** Fitur *download assignment* menghasilkan berkas dengan format penamaan *{id-assignment}-{nama-assignment}-{nama-problem}.zip*.

Tabel 3.1: Pemetaan FRS ke SRS

Kode FRS	Deskripsi Fitur Utama	Kode SRS	Spesifikasi Teknis (SRS)
FRS001	Fitur Pengelolaan Kuis & Latihan	**SRS001**	Fitur *flag* "quiz" / "latihan".
		SRS002	Kolom konfigurasi *IPv4 Subnets*.
		SRS003	Restriksi akses berdasarkan *IP Address*.
		SRS004	Ikon visualisasi tipe *assignment*.
		SRS005	Indikator warna hijau waktu aktif.
		SRS006	Penguncian *assignment* lain saat kuis aktif (Merah).
		SRS007	Akses *assignment* lain saat latihan aktif (Hitam).
		SRS008	Pesan kesalahan penolakan akses IP/Kuis.
FRS002	Pengunduhan Berkas Tugas Terstruktur	**SRS009**	Hak akses pengunduhan khusus *role Teacher*.
		SRS010	Penggabungan *.zip* hasil *Final Submissions*.
		SRS011	Format nama berkas *{id}-{nama}-{problem}.zip*.

3.3 Analisis Sistem Usulan

Pada pembuatan beberapa fitur baru pada SharIF-Judge diperlukan penambahan dan modifikasi pada beberapa modul utama, yaitu pemisahan kuis dan latihan melalui IPv4, pembatasan akses berdasarkan subnet IPv4 yang sudah diset, penguncian *assignment* pada saat kuis sedang berlangsung, format dalam mengunduh berkas jawaban setiap mahasiswa. Gambar pada usecase diagram menunjukkan bagaimana seluruh fitur baru akan saling berinteraksi dengan sistem dan pengguna.



Gambar 3.1: usecase analisis sistem usulan

3.3.1 Fitur Pemisahan Kuis dan Pembatasan Akses Subnet IPv4

Fitur pemisahan kuis dan pembatasan akses ini dirancang untuk mengatasi masalah penggunaan dua buah *instance* SharIF-Judge untuk kuis dan latihan. Fitur ini menggabungkan kedua *instance* menjadi satu sehingga latihan dan kuis dapat di kerjakan pada 1 tempat yang sama. Setiap *assignment* akan ditambahkan atribut penanda berupa (*flag*) tipe (kuis atau latihan) dan daftar subnet IPv4 yang diizinkan.

Ketika mahasiswa mencoba mengakses dan mengerjakan *assignment*, sistem akan mengecek alamat IPv4 milik mahasiswa, jika terdaftar pada subnet yang sudah disiapkan maka sistem akan melakukan proses validasi. Namun jika IPv4 tidak berada dalam batas subnet yang sudah disiapkan maka akan menampikkan pesan menolak akses yang dilakukan oleh mahasiswa tersebut.

Alur proses sistem dan interaksi pengguna dalam mengecek dan memvalidasi akses adalah sebagai berikut:

1. Mahasiswa memilih atau menekan salah satu *assignment* pada antarmuka peramban.
2. *View* mengirimkan permintaan ke controller *Assignments.php* melalui fungsi *select(assignment_id)*.
3. Controller meminta data *assignment* (termasuk atribut *flag quiz* dan konfigurasi *IPv4 subnets*) kepada *Assignment_model.php*.
4. *Assignment_model.php* melakukan kueri ke basis data dan mengembalikan data informasi *assignment* ke controller.

5. Controller mengambil alamat IP mahasiswa saat itu dan memvalidasinya menggunakan metode *validate_ip_subnet()* pada *User_model.php*.
6. Apabila alamat IP mahasiswa tidak sesuai dengan subnet yang diizinkan, controller menghentikan proses dan mengembalikan pesan penolakan yang informatif .
7. Apabila alamat IP sesuai, controller akan memeriksa aturan waktu dan status kuis aktif, lalu mengembalikan tampilan halaman *assignment* (*assignments.twig*) kepada mahasiswa.

3.3.2 Fitur Visualisasi Status dan Penguncian Assignment

itur ini dirancang untuk memberikan indikasi visual mengenai status *assignment* serta menjaga kerahasiaan kuis yang sedang berlangsung. Halaman daftar *assignment* akan menampilkan ikon penanda latihan atau kuis dan warna status waktu (SRS004, SRS005). Fitur ini berfungsi untuk memberikan suatu tampilan visual mengenai status suatu *assignment*. jika terdapat kuis yang sedang aktif dalam rentang waktu maka sistem akan mengunci assignment lain dan dan mewarnainya dengan warna merah, jika yang akif adalah tipe latihan, maka *assignment* lain akan tetap bisa di akses secara normal.

3.3.3 Fitur Pengunduhan Berkas

Fitur mengunduh berkas jawaban mahasiswa di perbaharui agar dosen mampi mengunduh seluruh jawaban terakhir secara bersaamaan. Sistem akan mengumpulkan seluruh hasil pengunduhan seluruh berkas dalam bentuk zip dengan format penamaan *{id-assignment}-{nama-assignment}-{nama-problem}.zip*

DAFTAR REFERENSI

[1] de Berg, M., Cheong, O., van Kreveld, M. J., dan Overmars, M. (2008) *Computational Geometry: Algorithms and Applications*, 3rd edition. Springer-Verlag, Berlin.

[2] van Kreveld, M. J. (2004) Geographic information systems. Bagian dari Goodman, J. E. dan O'Rourke, J. (ed.), *Handbook of Discrete and Computational Geometry*. Chapman & Hall/CRC, Boca Raton.

[3] Buchin, K., Buchin, M., van Kreveld, M. J., Löffler, M., Silveira, R. I., Wenk, C., dan Wiratma, L. (2013) Median trajectories. *Algorithmica*, **66**, 595–614.

[4] van Kreveld, M. J. dan Wiratma, L. (2011) Median trajectories using well-visited regions and shortest paths. *Proceedings of the 19th ACM SIGSPATIAL International Conference on Advances in Geographic Information Systems*, Chicago, USA, 1-4 November, pp. 241–250. ACM, New York.

[5] Lionov (2002) Animasi algoritma sweepline untuk membangun diagram voronoi. Skripsi. Universitas Katolik Parahyangan, Indonesia.

[6] Wiratma, L. (2010) Following the majority: a new algorithm for computing a median trajectory. Thesis. Utrecht University, The Netherlands.

[7] Wiratma, L. (2022) Coming Not Too Soon, Later, Delay, Someday, Hopefully. Disertasi. Utrecht University, The Netherlands.

[8] van kreveld, M., van Lankveld, T., dan Veltkamp, R. (2013) Watertight scenes from urban lidar and planar surfaces. Technical Report UU-CS-2013-007. Utrecht University, The Netherlands.

[9] Rekhter, Y. dan Li, T. (1994) A border gateway protocol 4 (bgp-4). RFC 1654. RFC Editor, http://www.rfc-editor.org.

[10] ITU-T Z.500 (1997) *Framework on formal methods in conformance testing*. International Telecommunications Union. Geneva, Switzerland.

[11] Version 9.0.0 (2016) *The Unicode Standard*. The Unicode Consortium. Mountain View, USA.

[12] Version 7.0 Nougat (2016) *Android API Reference Manual*. Google dan Open Handset Alliance. Mountain View, USA.

[13] Webb, R., Daruca, O., dan Alfadian, P. (2012) *Method of optimizing a text massage communication between a server and a secure element*. Paten no. EP2479956 (A1). European Patent Organisation. Munich, Germany.

[14] Wiratma, L. (2009) Median trajectory. Report for GMT Experimentation Project at Utrecht University.

[15] Lionov (2011) Polymorphism pada C++. Catatan kuliah AKS341 Pemrograman Sistem di Universitas Katolik Parahyangan, Bandung. http://tinyurl.com/lionov. 30 September 2016.

[16] Erickson, J. (2003) CG models of computation? http://www.computational-geometry.org/mailing-lists/compgeom-announce/2003-December/000852.html. 30 September 2016.

[17] AGUNG (2012) Menjajal tango 12. Majalah HAI no 02, Januari 2012.

LAMPIRAN A

KODE PROGRAM

Kode A.1: MyCode.c

```c

// This does not make algorithmic sense,
// but it shows off significant programming characters.

#include<stdio.h>

void myFunction( int input, float* output ) {
    switch ( array[i] ) {
        case 1: // This is silly code
            if ( a >= 0 || b <= 3 && c != x )
                *output += 0.005 + 20050;
            char = 'g';
            b = 2^n + ~right_size - leftSize * MAX_SIZE;
            c = (--aaa + &daa) / (bbb++ - ccc % 2 );
            strcpy(a,"hello $@?");
    }
    count = ~mask | 0x00FF00AA;
}

// Fonts for Displaying Program Code in LATEX
// Adrian P. Robson, nepsweb.co.uk
// 8 October 2012
// http://nepsweb.co.uk/docs/progfonts.pdf
```

Kode A.2: MyCode.java

```java
import java.util.ArrayList;
import java.util.Collections;
import java.util.HashSet;

//class for set of vertices close to furthest edge
public class MyFurSet {
    protected int id;                                   //id of the set
    protected MyEdge FurthestEdge;                      //the furthest edge
    protected HashSet<MyVertex> set;                    //set of vertices close to furthest edge
    protected ArrayList<ArrayList<Integer>> ordered;    //list of all vertices in the set for each trajectory
    protected ArrayList<Integer> closeID;               //store the ID of all vertices
    protected ArrayList<Double> closeDist;              //store the distance of all vertices
    protected int totaltrj;                             //total trajectories in the set

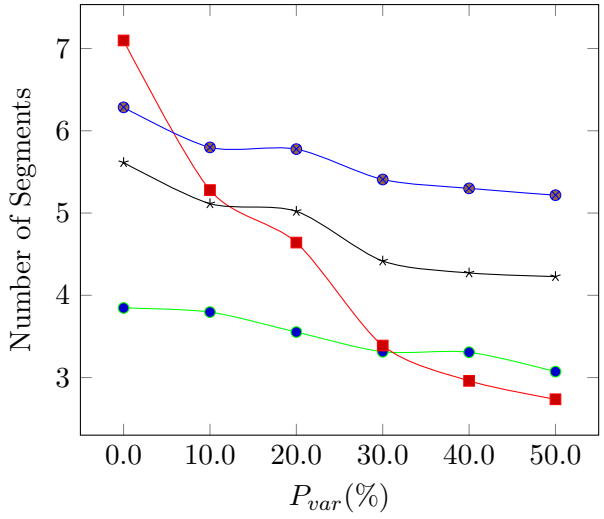
    /*
     * Constructor
     * @param id : id of the set
     * @param totaltrj : total number of trajectories in the set
     * @param FurthestEdge : the furthest edge
     */
    public MyFurSet(int id,int totaltrj,MyEdge FurthestEdge) {
        this.id = id;
        this.totaltrj = totaltrj;
        this.FurthestEdge = FurthestEdge;
        set = new HashSet<MyVertex>();
        ordered = new ArrayList<ArrayList<Integer>>();
        for (int i=0;i<totaltrj;i++) ordered.add(new ArrayList<Integer>());
        closeID = new ArrayList<Integer>(totaltrj);
        closeDist = new ArrayList<Double>(totaltrj);
        for (int i = 0;i <totaltrj;i++) {
            closeID.add(-1);
            closeDist.add(Double.MAX_VALUE);
        }
    }

}
```

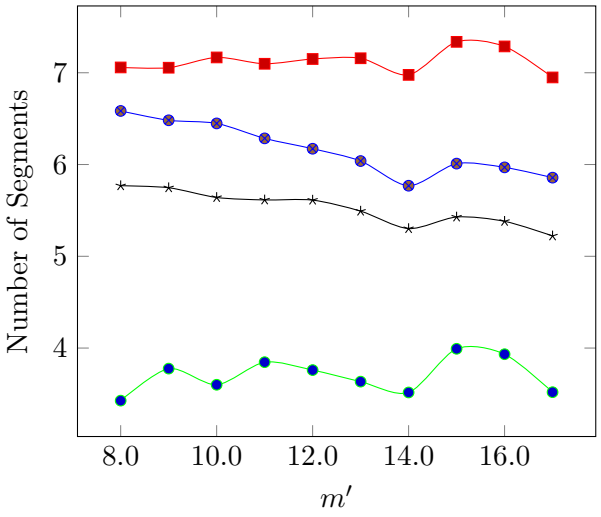
LAMPIRAN B

HASIL EKSPERIMEN

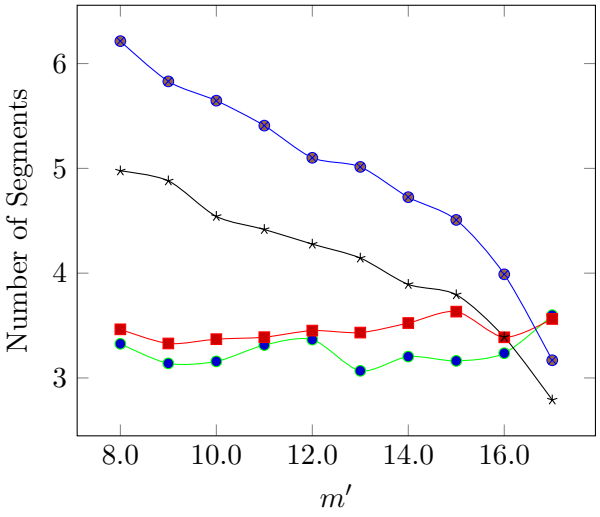
Hasil eksperimen berikut dibuat dengan menggunakan TIKZPICTURE (bukan hasil excel yg diubah ke file bitmap). Sangat berguna jika ingin menampilkan tabel (yang kuantitasnya sangat banyak) yang datanya dihasilkan dari program komputer.



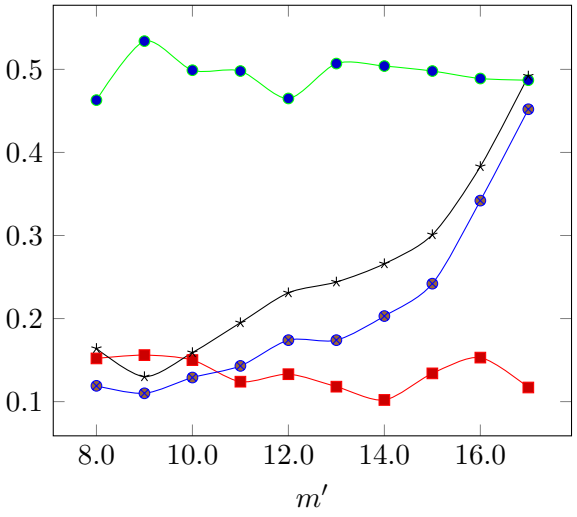
Gambar B.1: Hasil 1



Gambar B.2: Hasil 2



Gambar B.3: Hasil 3



Gambar B.4: Hasil 4